



ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
DEPARTMENT OF ELECTRICAL AND ELECTRONIC
ENGINEERING

Project Title: Digital Alarm Clock

Group No : B2 (Group Two)

Student Information:

1. Faiaz Riasat Saad – 200021204
2. Sayeed Hossain Sayor – 200021236
3. M. M. Sadman Shafi – 200021240
4. Shafayet Sadik Sowad – 200021248
5. Iftikhar Azad – 200021254
6. Were Shuraim – 200021252

Department: EEE

Section: B

Course: EEE 4706 (Microcontroller and System Design Lab)

Objective:

The objective of this project is to design and implement a Digital Alarm Clock that not only displays time and alarms but also incorporates interactive and engaging features such as multi-alarm support, customizable snooze durations, and an innovative LED-based alarm deactivation system to ensure active user participation when dismissing alarms.

Goals:

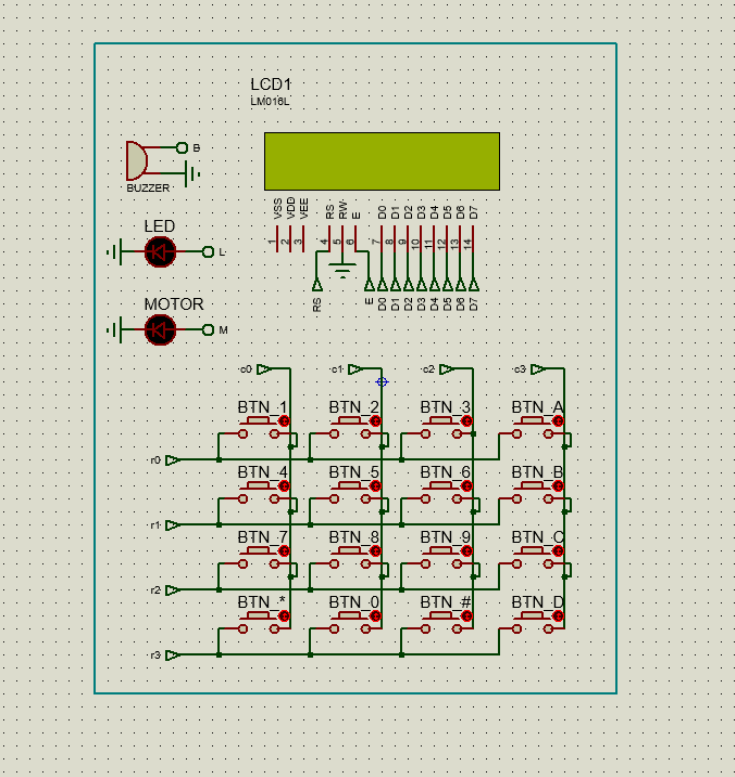
The goal is to create a user-friendly digital alarm clock that:

1. Allows users to set and manage multiple alarms easily.
2. Provides flexible snooze options (5, 10, or 15 minutes) based on user preference.
3. Enhances alarm dismissal by requiring users to input a specific key pattern according to randomly lit LEDs, thus ensuring the user is awake and alert.
4. Integrating a loud buzzer, a clear digital display, and a responsive keypad for effective interaction.

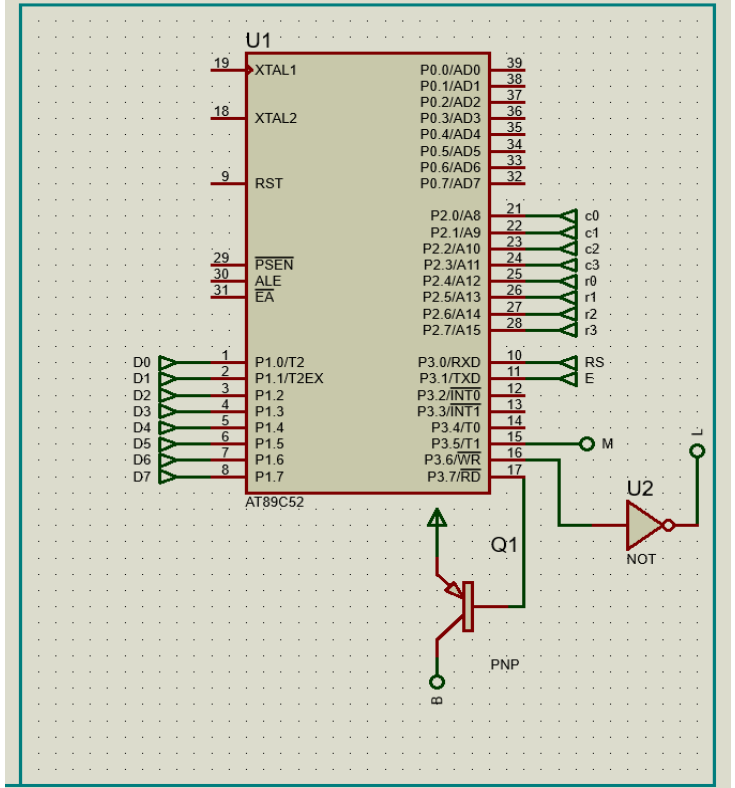
Required Components:

Component	Price
Microcontroller	5000
Keypad	Built-in
Jumper Wires	200
Buzzer	Built-in
LED	Built-in
LCD	Built-in
Motor	1500
MOSFET	50
Total	6750 BDT

Circuit Diagram:



Schematic



Explanation of Circuit Diagram:

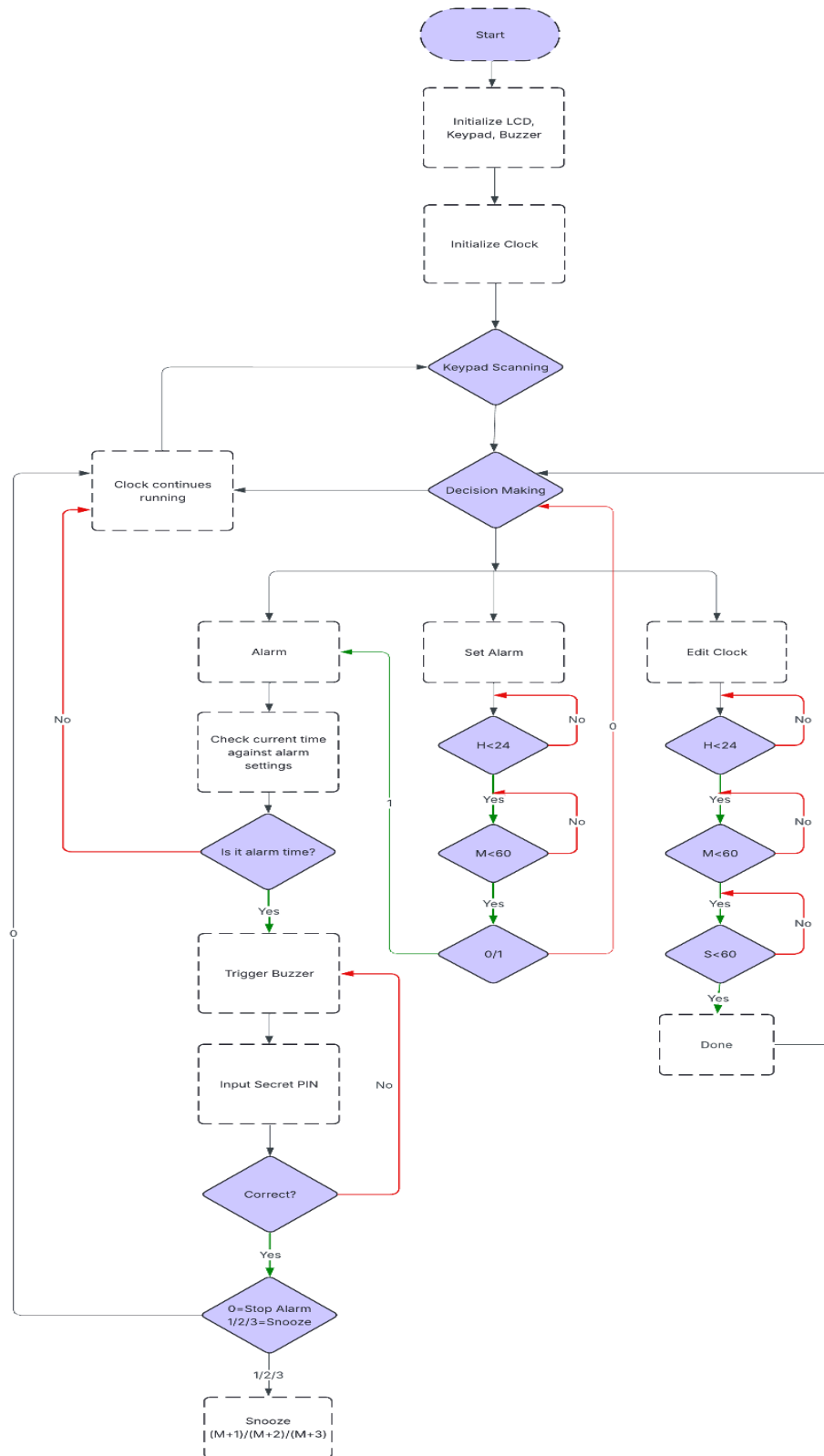
- Port 1 sends the Microcontroller data to LCD
- Port 2 is connected to the Keypad. Rows: P2.0 to P2.3 (lower nibble), Columns: P2.4 to P2.7 (Upper Nibble)
- Pin 3.7 connected to Buzzer
- Pin 3.6 connected to Alarm Indicator LED.
- Pin 3.5 connected to Pump Motor (represented by LED)

Features:

- **Clock Mode**
The clock continuously displays the current time on the LCD screen, updating every second for accurate real-time tracking. The display remains active at all times, ensuring the user can easily view the current time at a glance.
- **Alarm Setting Mode**
 - Users can set alarm times by inputting hours and minutes through the keypad.
 - An alarm icon appears on the LCD to indicate when an alarm is active.
- **Alarm Trigger Mode**
 - A loud buzzer sounds to alert the user.
 - A motor-driven system sprays water to ensure the user wakes up actively. The water-spraying motor acts as a physical stimulus to prevent the user from ignoring the alarm.
 - Alarm indicating LED
- **Alarm Stopping Mode**
 - Enter a preset secret key, adding an interactive challenge. If the correct key is not entered, the alarm will continue to sound.
 - Fully deactivate the alarm
 - Snooze alarm. Snooze durations are customizable with options of 1, 2, or 3 minutes.
- **Edit Time:**

Edit Time Mode lets the user manually set the clock by entering hours, minutes, and seconds. It checks the inputs to make sure they're valid and updates the system timers. After editing, the clock runs with the new time.

Flowchart:



Working Principle:

Step-by-step explanation of the general process:

1. Initialization Phase:

The program is started by setting up an initialization value for some of the modules:

- a. LCD: Used to display time, alarm settings, questions, etc.
- b. Keypad: To receive user inputs such as numbers, modes, etc.
- c. Timer1: To generate 10 mili-second time intervals using interrupts.
- d. Variables: Time variables such as REALSCND, REALMINT, REALHOUR, and alarm time variables are set to 0.

2. Main Loop Operation:

The code enters an infinite while loop where it continuously does the following functions:

- a. Scans the keypad to detect input for any instructions.
- b. Updates the display continuously with the values of real time second, minute and hour.
- c. It checks and compares the alarm time with the main clock minute and hour values so that it can decide if the alarm should ring.

3. Mode Switching:

The program consists of 4 different modes. They are explained below:

- a. Mode 1: Clock Display Mode:
 - It is the default mode of the clock
 - The current time is shown on the LCD while this mode is active
 - The clock is updated here every second.
- b. Mode 2: Edit Time Mode:
 - This mode can be used to set the time manually
 - Can also be used to change the time to desired value
 - Hour, Minute and Second can be freely changed
 - After saving the desired time, the mode is automatically switched to Clock Display Mode
- c. Mode 3: Alarm Set Mode:
 - Allows the user to set alarm manually
 - Only Hour and Minute can be set
 - After setting the alarm, there is a symbol 'A' that is present on the top right of Clock Display Mode.
 - After setting the alarm, the user is automatically taken to the Clock Display Mode.

d. Mode 4: Alarm Resolve Mode:

- When the current time on the Clock Display Mode reaches the set alarm time, Alarm Resolve Mode is activated
- In this mode, the buzzer starts making sound, and there is a connected motor that starts spilling small amounts of water to alert the user.
- In the next window, the display asks for a secret key to let you enter the next stage.
- If the user fails to provide the secret key, then the alarm keeps ringing.
- After providing the secret key, the user can either select 0 to stop the alarm or enter 3 modes of snooze.
- In the snooze mode option, user has the option to set the snooze timer at 1, 2 or 3 minutes, which starts from the set alarm time

4. Timer Interrupt Handling:

The timer is configured to send an interrupt every 10 milisecond. When an interrupt occurs, the following actions take place:

- a. After each iteration, REALMSCND is incremented by 1.
- b. After REALMSCND is incremented to 100, the second counter REALSCND is incremented by 1. REALMSCND is reset to 0
- c. When the second counter reaches 60
 - The second counter is reset to 0
 - The minute counter is incremented by 1
- d. Similarly when the minute counter reaches 60
 - The minute counter is reset to 0
 - The hour counter is incremented by 1
- e. Timer ensures that the Real Time Clock can function without any error
- f. The hardware ensures that precise interrupt timing is achieved without error

5. Keypad Input Handling:

The Keypad Input Handling part ensures that the user inputs are constantly checked and the instructions are done accordingly. Moreover, the following actions are also conducted:

- a. In the main loop, input keys are constantly checked for user instructions
- b. When a key is pressed, all the rows and columns are checked and the corresponding key index is identified and key is stored in KEYMEM.
- c. After identifying the key, the program checks for the task related to the key in KEYMEM and performs accordingly
- d. The Keypad Input Handling has the following keys in this program:
 - (0-9) keys for the digits

- * key for Alarm Set function
- D key for Edit Time function

Code:

; PIN DEFINATIONS

; LCD

```
LCD_PORT    EQU        P1
LCD_RS      EQU        P3.0
LCD_ENABLE  EQU        P3.1
```

;KEYPAD

```
KEY_PORT    EQU        P2
```

```
MOV P0, #00H
```

;BUZZER

```
BUZZER      EQU        P3.7
```

;LED SET

```
ALRMSET     EQU        P3.6
```

; DATA DEFINATIONS

```
KEYMEM      EQU        024H
REALMSCND   EQU        026H
REALSCND    EQU        027H
REALMINT    EQU        028H
REALHOUR    EQU        029H
CLR C
ALRMMINT    EQU        02AH
ALRMHOUR    EQU        02BH
ISALRM      EQU        02CH
ISSNZ       EQU        02DH
MODE        EQU        02EH
SOLUTION    EQU        02FH
ACTUAL      EQU        030H
BUFF        EQU        032H
```

; TEXT

```
ORG         0000H
LJMP  MAIN

ORG         001BH
LJMP  TIMER1ISR

ORG         0030H
```

MAIN:

```
SETB  BUZZER
MOV    SP,          #55H      ; MOVING SP TO SCRATCHPAD
MOV    KEYMEM,      #' '
MOV    MODE,        #01H ;default mode 1. Displays the clock
MOV    ALRMHOUR,    #00H
MOV    ALRMMINT,    #00H
MOV    ISALRM,      #00H
MOV    ISSNZ,       #00H
MOV    REALHOUR,    #00
MOV    REALMINT,    #00
MOV    REALSCND,    #00
CLR    P3.5
```

LCALL SETUPTIMERS ; Timer 0 and 1 set to count 10ms in Mode 1. Interrupt Reg used to set Timer 1 and Timer 0 overflow

```
LCALL DELAY
LCALL DELAY
LCALL LCD_INIT
LCALL KEYPAD_INIT
```

```
SETB  TR1
```

MAINLOOP:

```
LCALL KEYPAD_SCAN
```

```
MOV    C,          ISALRM.0
MOV    ALRMSET,    C
```

; check if alarm time

```
MOV    A,          ISALRM
JZ     SKIP_ALARM
```

```
MOV    A,          REALMINT
XRL    A,          ALRMMINT
JNZ    SKIP_ALARM
```

```
MOV    A,          REALHOUR
XRL    A,          ALRMHOUR
JNZ    SKIP_ALARM
```

```
MOV    C,          ISALRM.0
CPL    C
MOV    ALRMSET,    C
```

MOV MODE, #04H ;If clock time and alarm time does not match, mode 4 will be active. This will activate the alarm

SKIP_ALARM:

```
MOV    A,          KEYMEM
XRL    A,          #'D'
JZ     CHANGEMODE
```

```
MOV    A,          KEYMEM
XRL    A,          #'*'
JZ     ALRM_MODE
```

```
SJMP  DONE
```

ALRM_MODE:

```
MOV    MODE,      #03H
LJMP  CHANGEMODE_DONE
```

CHANGEMODE:

```
MOV    A,          MODE
XRL    A,          #01H
JZ     GO_TO_EDIT
```

GO_TO_EDIT:

```
MOV    MODE,      #02H
```

CHANGEMODE_DONE:

```
MOV    A,          #01H
LCALL LCD_COMMAND
LCALL DELAY
SJMP  DONE
```

DONE:

```
MOV    A,          MODE
XRL    A,          #01H ; default working mode. displays CLOCK
on first line. Displays A is there is alarm. Then on next line, displays REALMINT, REALHOUR,
REALSCND. Uses NUM2ASCII to covert minute and second to ascii values
JZ     CLOCK

MOV    A,          MODE
XRL    A,          #02H ;Once D is pressed, it will clear the existing
screen and jump tp EDITTIME
```

```

JZ          EDITTIME

MOV         A,          MODE
XRL         A,          #03H
JZ          ALRMMODE

MOV         A,          MODE
XRL         A,          #04H
JZ          ALRMRESOLVE

LJMP  MAINLOOP

```

CLOCK:

```

LCALL DISPLAY_CLOCK
LJMP  MAINLOOP

```

EDITTIME:

```

CLR         TR1
LCALL DISPLAY_EDITTIME
LCALL KEY_TAKE_TIME
LCALL SETUPTIMERS
MOV         MODE, #01H
MOV         A,    #01H
LCALL LCD_COMMAND
LCALL DELAY
SETB  TR1
LJMP  MAINLOOP

```

ALRMMODE:

```

LCALL DISPLAY_ALRMTIME
LCALL KEY_TAKE_ALARM
MOV  MODE, #01H
MOV  A,    #01H
LCALL LCD_COMMAND
LCALL DELAY
LJMP  MAINLOOP

```

ALRMRESOLVE:

```

CLR  BUZZER
SETB P3.5
LCALL DISPLAY_ALRMRESOLVE
LCALL KEY_TAKE_SOL

MOV         A,          #01H
LCALL LCD_COMMAND
LCALL         DELAY

```

```
LCALL DISPLAY_SNOOZE
LCALL KEY_TAKE_SNOOZE
```

```
SETB  BUZZER
CLR P3.5
MOV  MODE, #01H
MOV  A,    #01H
LCALL LCD_COMMAND
LCALL    DELAY
LCALL    MAINLOOP
```

DISPLAY_SNOOZE:

```
MOV      A,                #80H
LCALL LCD_DISPLAY

MOV      DPTR,             #STEXT
LCALL COPYBUFFER
LCALL LCD_DISPLAYS

RET
```

DISPLAY_ALRMRESOLVE:

```
MOV  A,    #01H
LCALL LCD_COMMAND

MOV      A,                #80H
LCALL LCD_COMMAND

MOV      DPTR,             #EQN
LCALL COPYBUFFER
LCALL LCD_DISPLAYS

MOV      A,                #0C0H
LCALL LCD_COMMAND
MOV      ACTUAL,           #07

RET
```

DISPLAY_ALRMTIME:

```
MOV      A,                #0CH
LCALL LCD_COMMAND

MOV      A,                #80H
LCALL LCD_COMMAND

MOV      DPTR,             #ATIME
LCALL COPYBUFFER
LCALL LCD_DISPLAYS
```

MOV A, #0C0H
LCALL LCD_COMMAND

MOV DPTR, #AHOLDER
LCALL COPYBUFFER
LCALL LCD_DISPLAYS

RET

DISPLAY_EDITTIME:

MOV A, #0CH
LCALL LCD_COMMAND

MOV A, #80H
LCALL LCD_COMMAND

MOV DPTR, #ETIME
LCALL COPYBUFFER
LCALL LCD_DISPLAYS

MOV A, #0C0H
LCALL LCD_COMMAND

MOV DPTR, #PHOLDER
LCALL COPYBUFFER
LCALL LCD_DISPLAYS

RET

DISPLAY_CLOCK:

MOV A, #0CH
LCALL LCD_COMMAND

MOV A, #80H
LCALL LCD_COMMAND

MOV DPTR, #CLK
LCALL COPYBUFFER
LCALL LCD_DISPLAYS

MOV A, #' '
LCALL LCD_DISPLAY

MOV A, KEYMEM
LCALL LCD_DISPLAY

MOV A, #8FH
LCALL LCD_COMMAND

```

MOV      A,          ISALRM
JZ       SKIP_ALARM_PRINT

MOV      A,          #'A'
LCALL LCD_DISPLAY

SKIP_ALARM_PRINT:

MOV      A,          #0C0H
LCALL LCD_COMMAND

MOV      A,          REALHOUR
LCALL NUM2ASCII
LCALL LCD_DISPLAYS

MOV      A,          #':'
LCALL LCD_DISPLAY

MOV      A,          REALMINT
LCALL NUM2ASCII
LCALL LCD_DISPLAYS

MOV      A,          #':'
LCALL LCD_DISPLAY

MOV      A,          REALSCND
LCALL NUM2ASCII
LCALL LCD_DISPLAYS

LCALL DELAY

RET

;----- Utilities -----;

COPYBUFFER:
PUSH 00H
PUSH 01H
PUSH 07H

MOV R0, #00H
MOV R1, #BUFF

COPYBUFFER_LOOP:
MOV A, R0
MOVC A, @A+DPTR ;Whatever is stored in DPTR, is moved to A,
which is then moved to BUFF location using R1
MOV @R1, A
JZ COPYBUFFER_DONE

```

```

        INC            R0
        INC            R1
        SJMP  COPYBUFFER_LOOP
COPYBUFFER_DONE:
        POP            07H
        POP            01H
        POP            00H
        RET

```

NUM2ASCII:

```

        PUSH  00H

        MOV    R0,          #BUFF
        MOV    B,          #10
        DIV    AB

        ADD    A,          #30H
        MOV    @R0,        A
        INC    R0
        MOV    A,          B

        ADD    A,          #30H
        MOV    @R0,        A
        INC    R0

        MOV    @R0,        #00H

        POP    00H
        RET

```

;----- LCD Code -----;

LCD_INIT:

```

        MOV    LCD_PORT,   #00H
        CLR    LCD_PORT
        CLR    LCD_ENABLE
        CLR    LCD_RS

        MOV    A,          #38H
        LCALL LCD_COMMAND

        MOV    A,          #0CH
        LCALL LCD_COMMAND

        MOV    A,          #06H
        LCALL LCD_COMMAND

        MOV    A,          #01H
        LCALL LCD_COMMAND

```

```
MOV      A,          #80H
LCALL LCD_COMMAND
```

```
RET
```

LCD_DISPLAYS:

```
PUSH 00H
```

```
MOV      R0,          #BUFF ;contents of BUFF are moved to A to be displayed
```

LCD_DISPLAYS_LOOP:

```
MOV      A,          @R0
```

```
JZ      LCD_DISPLAYS_DONE
```

```
LCALL LCD_DISPLAY
```

```
INC      R0
```

```
SJMP LCD_DISPLAYS_LOOP
```

LCD_DISPLAYS_DONE:

```
POP      00H
```

```
RET
```

LCD_DISPLAY:

```
LCALL LCD_WAIT
```

```
SETB LCD_RS
```

```
MOV      LCD_PORT,  A
```

```
SETB LCD_ENABLE
```

```
NOP
```

```
CLR      LCD_ENABLE
```

```
NOP
```

```
RET
```

LCD_COMMAND:

```
LCALL LCD_WAIT
```

```
CLR      LCD_RS
```

```
MOV      LCD_PORT,  A
```

```
SETB LCD_ENABLE
```

```
NOP
```

```
CLR      LCD_ENABLE
```

```
NOP
```

```
RET
```

LCD_WAIT:

```
PUSH 07H
```

```
MOV      R7,          #0FFH
```

```
LCALL SIMPLE_DELAY
```

```
MOV      R7,          #0FFH
```

```
LCALL SIMPLE_DELAY
POP      07H
RET
```

;----- DELAYS -----;

```
SIMPLE_DELAY:
    DJNZ  R7,      SIMPLE_DELAY
    RET
```

```
DELAY:
    PUSH  03H
    PUSH  07H
    MOV   R3,      #255D
AGAIN:
    MOV   R7,      #255D
    ACALL SIMPLE_DELAY
    DJNZ  R3,      AGAIN

    POP   07H
    POP   03H
    RET
```

; ----KEYPAD-----

```
KEYPAD_INIT:
    MOV   KEY_PORT, #0FFH
    RET
```

```
KEYPAD_IDX:
    PUSH  00H
    MOV   R0,      A

    MOV   A,      R0
    XRL   A,      #00001110B
    JZ    KEYPAD_IDX_ZERO

    MOV   A,      R0
    XRL   A,      #00001101B
    JZ    KEYPAD_IDX_ONE

    MOV   A,      R0
    XRL   A,      #00001011B
    JZ    KEYPAD_IDX_TWO

    MOV   A,      R0
    XRL   A,      #00000111B
    JZ    KEYPAD_IDX_THREE
```

```

        MOV      A,          #04H
        SJMP     KEYPAD_IDX_DONE

KEYPAD_IDX_ZERO:
        MOV      A,          #00H
        SJMP     KEYPAD_IDX_DONE

KEYPAD_IDX_ONE:
        MOV      A,          #01H
        SJMP     KEYPAD_IDX_DONE

KEYPAD_IDX_TWO:
        MOV      A,          #02H
        SJMP     KEYPAD_IDX_DONE

KEYPAD_IDX_THREE:
        MOV      A,          #03H

KEYPAD_IDX_DONE:
        POP      00H
        RET

KEYPAD_SCAN:
        PUSH     00H
        PUSH     01H
        PUSH     07H

        MOV      KEY_PORT,   #0F0H
        MOV      A,          KEY_PORT
        ANL      A,          #0F0H
        SWAP     A
        LCALL    KEYPAD_IDX
        MOV      R0,         A

        XRL      A,          #04H
        JZ       KEY_SCAN_DONT

        MOV      KEY_PORT,   #00FH
        MOV      A,          KEY_PORT
        ANL      A,          #00FH
        LCALL    KEYPAD_IDX
        MOV      R1,         A

        XRL      A,          #04H
        JZ       KEY_SCAN_DONT

        MOV      A,          R0
        RL       A
        RL       A

```

```

        ADD      A,          R1
        MOV      DPTR,      #KEYPADBUF

        MOVC A,          @A+DPTR
        MOV      KEYMEM,    A
        SJMP     KEY_SCAN_DONE

KEY_SCAN_DONT:
        MOV      KEYMEM,    #' '

KEY_SCAN_DONE:
        POP      07H
        POP      01H
        POP      00H
        RET

KEY_TAKE_SNOOZE:
        MOV      A,          #0EH
KEY_SNZ:
        MOV      A,          #0C0H
        LCALL   LCD_COMMAND

        MOV      A,          #'?'
        LCALL   LCD_DISPLAY

        MOV      A,          #0C0H
        LCALL   LCD_COMMAND

        LCALL   KEY_TAKE_ONE_DIGIT
        MOV      B,          A
        CLR      C
        SUBB     A,          #04D
        LCALL   DELAY
        JNC      KEY_SNZ

        MOV      A,          B
        JZ       ALRM_STOP

        MOV      ISSNZ, #01H

        LCALL   DELAY
        MOV      B, A
        XRL      A,          #1
        JZ       SNZ1
        MOV      A, B

        XRL      A,          #2
        JZ       SNZ2
        MOV      A, B

```

```
        XRL      A,          #3
        JZ       SNZ3
```

SNZ1:

```
        MOV      A,          ALRMMINT
        ADD      A,          #01D
        MOV      ALRMMINT, A
        SJMP     HERE
```

SNZ2:

```
        MOV      A,          ALRMMINT
        ADD      A,          #2D
        MOV      ALRMMINT, A
        SJMP     HERE
```

SNZ3:

```
        MOV      A,          ALRMMINT
        ADD      A,          #3D
        MOV      ALRMMINT, A
```

HERE: RET

ALRM_STOP:

```
        MOV      ISSNZ, #00H
        MOV      ISALRM, #00H
        RET
```

KEY_TAKE_SOL:

```
        MOV      A,          #0EH
        LCALL    LCD_COMMAND
```

KEY_SOLVE:

```
        MOV      A,          #0C6H
        LCALL    LCD_COMMAND
```

```
        MOV      A,          #'?'
        LCALL    LCD_DISPLAY
        LCALL    LCD_DISPLAY
```

```
        MOV      A,          #0C6H
        LCALL    LCD_COMMAND
```

```
        MOV      A,          #SOLUTION
        LCALL    KEY_TAKE_ONE_NUMBER
```

```
        MOV      A,          SOLUTION
        XRL      A,          ACTUAL
```

LCALL DELAY

JNZ KEY_SOLVE

RET

KEY_TAKE_ALARM:

MOV A, #0EH
LCALL LCD_COMMAND

KEY_ALRMHOUR:

MOV A, #0C0H
LCALL LCD_COMMAND

MOV A, #'-'
LCALL LCD_DISPLAY
LCALL LCD_DISPLAY

MOV A, #0C0H
LCALL LCD_COMMAND

MOV A, #ALRMHOUR
LCALL KEY_TAKE_ONE_NUMBER

MOV A, ALRMHOUR
CLR C
SUBB A, #25D

LCALL DELAY

JNC KEY_ALRMHOUR

MOV A, #':'
LCALL LCD_DISPLAY

KEY_ALRMMINT:

MOV A, #0C3H
LCALL LCD_COMMAND

MOV A, #'-'
LCALL LCD_DISPLAY
LCALL LCD_DISPLAY

MOV A, #0C3H
LCALL LCD_COMMAND

MOV A, #ALRMMINT
LCALL KEY_TAKE_ONE_NUMBER

```
MOV      A,      ALRMMINT
CLR      C
SUBB  A,      #61D
```

```
LCALL DELAY
```

```
JNC      KEY_ALRMMINT
```

```
KEY_ALARMSET:
```

```
MOV      A,      #0C6H
LCALL LCD_COMMAND
```

```
MOV      A,      #'?'
LCALL LCD_DISPLAY
```

```
MOV      A,      #0C6H
LCALL LCD_COMMAND
```

```
LCALL KEY_TAKE_ONE_DIGIT
```

```
MOV      B,      A
```

```
CLR      C
```

```
SUBB  A,      #02D
```

```
LCALL DELAY
```

```
JNC      KEY_ALARMSET
```

```
MOV      ISALRM,      B
```

```
RET
```

```
KEY_TAKE_TIME:
```

```
MOV      A,      #0EH
LCALL LCD_COMMAND
```

```
KEY_REALHOUR:
```

```
MOV      A,      #0C0H
LCALL LCD_COMMAND
```

```
MOV      A,      #'-'
LCALL LCD_DISPLAY
LCALL LCD_DISPLAY
```

```
MOV      A,      #0C0H
LCALL LCD_COMMAND
```

```
MOV      A,      #REALHOUR
LCALL KEY_TAKE_ONE_NUMBER
```

```
MOV      A,      REALHOUR
CLR      C
```

SUBB A, #25D

LCALL DELAY

JNC KEY_REALHOUR

MOV A, #':'

LCALL LCD_DISPLAY

KEY_REALMINT:

MOV A, #0C3H

LCALL LCD_COMMAND

MOV A, #'-'

LCALL LCD_DISPLAY

LCALL LCD_DISPLAY

MOV A, #0C3H

LCALL LCD_COMMAND

MOV A, #REALMINT

LCALL KEY_TAKE_ONE_NUMBER

MOV A, REALMINT

CLR C

SUBB A, #61D

LCALL DELAY

JNC KEY_REALMINT

MOV A, #':'

LCALL LCD_DISPLAY

KEY_REALSCND:

MOV A, #0C6H

LCALL LCD_COMMAND

MOV A, #'-'

LCALL LCD_DISPLAY

LCALL LCD_DISPLAY

MOV A, #0C6H

LCALL LCD_COMMAND

MOV A, #REALSCND

LCALL KEY_TAKE_ONE_NUMBER

MOV A, REALSCND

```

CLR          C
SUBB  A,     #61D

LCALL DELAY

JNC          KEY_REALSCND

RET

```

KEY_TAKE_ONE_NUMBER:

```

PUSH 00H
PUSH 01H
MOV   R0,    A

LCALL KEY_TAKE_ONE_DIGIT
MOV   B,     #10D
MUL   AB
MOV   R1,    A
LCALL KEY_TAKE_ONE_DIGIT
ADD   A,     R1

MOV   @R0,   A

POP   01H
POP   00H
RET

```

KEY_TAKE_ONE_DIGIT:

```

PUSH 07H
PUSH 02H
PUSH 01H

```

KEY_NO_INPUT_DEBOUNCE:

```

LCALL KEYPAD_SCAN
MOV   A,     KEYMEM
XRL   A,     #' '
JZ    KEY_NO_INPUT_DEBOUNCE
LCALL KEYPAD_SCAN
MOV   A,     KEYMEM
LCALL DELAY
LCALL KEYPAD_SCAN
XRL   A,     KEYMEM
JNZ   KEY_NO_INPUT_DEBOUNCE

```

```

MOV   B,     KEYMEM
MOV   R2,    #10H
MOV   R1,    #'0'

```

KEY_CHECK_NUMBER:

```

MOV   A,     B

```

```

        XRL            A,      R1
        JZ             KEY_IS_NUMBER
        INC            R1
        DJNZ R2,      KEY_CHECK_NUMBER
KEY_NOT_NUMBER:
        SJMP KEY_NO_INPUT_DEBOUNCE
KEY_IS_NUMBER:
        MOV            A,      B
        LCALL LCD_DISPLAY
        XRL            A,      #'0'

        POP            01H
        POP            02H
        POP            07H
        RET

; -----TIMERS-----
SETUPTIMERS:
        MOV            TMOD,    #10H
        MOV            TH1,     #0DBH
        MOV            TL1,     #0F0H
        MOV            IE,      #10001000B
        RET

TIMER1ISR:
        CLR            TR1
        PUSH 0D0H
        PUSH 0E0H
        PUSH 0F0H

        INC            REALMSCND
; Update real time
; SECOND CALCULATION
        MOV            A,      REALMSCND
        MOV            B,      #100D
        DIV            AB
        MOV            REALMSCND, B
        ADD            A,      REALSCND
        MOV            REALSCND, A

; MINUTE CALCULATION
        MOV            B,      #60D
        DIV            AB
        MOV            REALSCND, B
        ADD            A,      REALMINT
        MOV            REALMINT, A

```

; HOUR CALCULATION

```
MOV     B,                #60D
DIV     AB
MOV     REALMINT,         B
ADD     A,                REALHOUR
MOV     REALHOUR,         A
MOV     TH1,              #0DCH
MOV     TL1,              #000H
```

```
MOV     A,                MODE
XRL     A,                #04H
JNZ     TIMER1ISR_DONE
```

```
MOV     A,                ISSNZ
JNZ     TIMER1CONT
```

```
MOV     A,                REALSCND
SJMP    TIMER1ISR_DONE
```

TIMER1CONT:

```
CLR     BUZZER
```

TIMER1ISR_DONE:

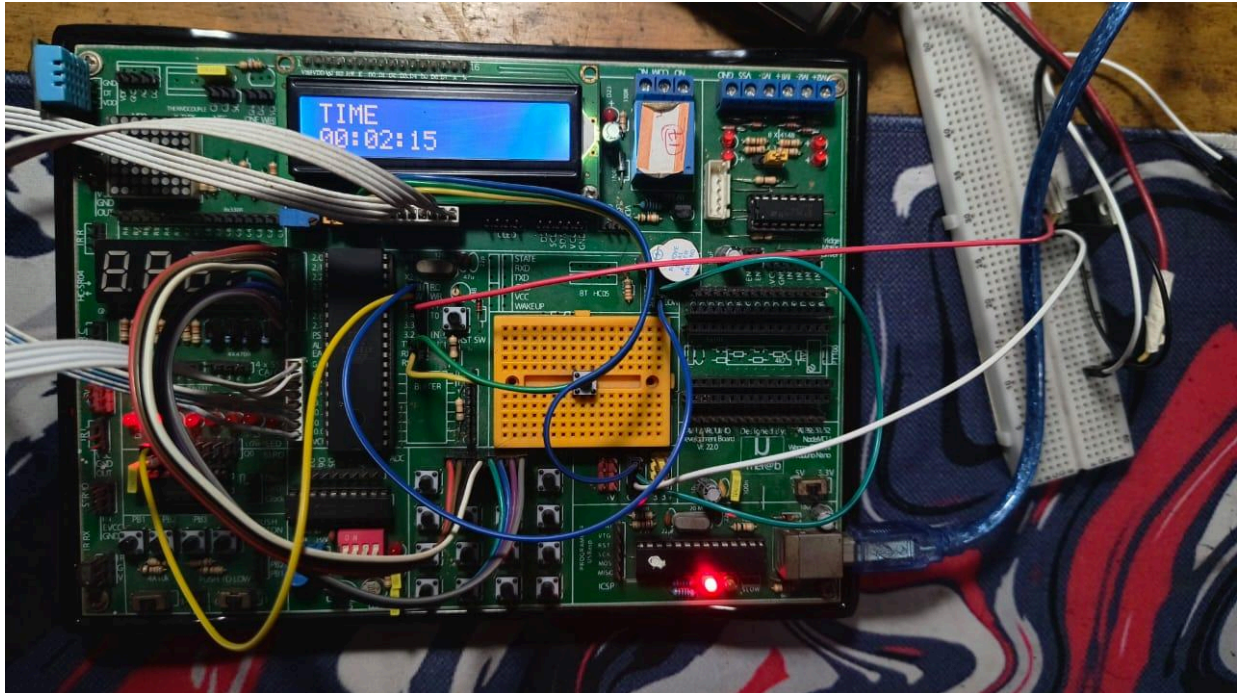
```
POP     0F0H
POP     0E0H
POP     0D0H
SETB    TR1
RETI
```

; DATA

```
CLK:    DB    "TIME", 0
ETIME:  DB    "EDIT TIME", 0
ATIME:  DB    "ALRM HH:MM (0/1)", 0
EQN:    DB    "ENTER SCRT KEY", 0
PHOLDER: DB    " : : ", 0
AHOLDER: DB    " : ? ", 0
STEXT:  DB    "STP-0/SNZ-1/2/3", 0
KEYPADBUF:
        DB    "123A"
        DB    "456B"
        DB    "789C"
        DB    "*0#D"
```

END

Hardware Implementation:



Problems Faced:

Software:

- It was really difficult to take inputs for enabling Alarm Setting mode while the timer was running and the clock was continuously being displayed on the LCD.
- Another problem we faced was the continuous display of the real-time values, which was not updating as expected.
- While trying to input digits into the keypad, there was a debouncing issue that caused double input.
- We tried to implement multiple alarms at once but while executing the second alarm time, it wasn't getting triggered properly.
- The instructed LED puzzle solving feature was too complex for us to implement, so we opted for a secret key input feature.
- Garbage/residual value was being shown on the LCD display.

Hardware:

- The Microcontroller board wasn't generating enough power to run the motor on the P3.5.
- The Port 1 of the Microcontroller board was faulty, so we had to use the port 0 instead of that in the hardware section.

- The connection of the burning switch of the Microcontroller board was a bit loose, causing multiple failed attempts of uploading the code to the board.

Conclusion:

The Digital Alarm Clock project successfully demonstrates the integration of hardware and software to create a functional and interactive timekeeping device. The core objectives – displaying real-time, setting alarms, customizable snooze options, and an engaging alarm deactivation system were achieved, although with some modifications to accommodate technical constraints.

Delivered Outcomes:

1. **Real-Time Clock Functionality:** The system accurately tracks and displays time using Timer 1 interrupts, with seamless updates every second. Users can manually edit hours, minutes, and seconds via the keypad.
2. **Alarm System:** Alarms trigger both auditory (buzzer) and physical (motor-driven water spray) stimuli, ensuring effective wake-up alerts. The snooze feature allows delays of 1, 2, or 3 minutes, enhancing user flexibility.
3. **Interactive Deactivation:** While the original LED puzzle concept was simplified to a secret key input due to complexity, the system ensures user engagement by requiring a valid key to stop or snooze alarms.

Areas for Improvement:

- Implementing multiple alarms would require enhanced memory management and interrupt prioritization.
- A dedicated debouncing circuit or algorithm could further stabilize keypad inputs.
- Restoring the LED puzzle feature can give an added benefit to the interactivity of the project.
- Power management upgrades, such as external drivers for the motor, would ensure proper effectiveness of the alarm system.

This project demonstrated the value of iterative testing, flexibility in problem-solving, and striking a balance between technical viability and ambition. It gave practical experience with peripheral interface, microcontroller programming, and debugging under real-world conditions.

In summary, even though the project achieved its main objectives, it also revealed areas that needed improvement, laying the groundwork for upcoming functional and user experience upgrades. From idea to execution, the process has been priceless, encouraging creativity and reaffirming important engineering concepts.